

Exercice 3 — Thread-safety : HashMap vs ConcurrentHashMap

EILMS — Java Enterprise (Step 1 & Step 2)

Énoncé

Pourquoi ce code peut planter (ou produire des résultats incohérents) avec un HashMap, mais pas avec un ConcurrentHashMap ?

```
Map<String, WarehouseItem> storage = new HashMap<>();  
// 10 threads exécutent en parallèle :  
storage.put("ITEM-" + i, new WarehouseItem(...));
```

Correction

HashMap n'est pas thread-safe en interne : sa structure de données (tableau de buckets, listes/arbres chaînés) n'est pas protégée contre les accès concurrents. Si deux threads écrivent en même temps dans le même bucket, ça peut provoquer :

- Des données perdues (une écriture écrase l'autre silencieusement)
- Une boucle infinie (bug historique connu lors du redimensionnement concurrent du tableau interne)
- Une `ConcurrentModificationException` si un thread lit pendant qu'un autre modifie

`ConcurrentHashMap` segmente les verrous au niveau des buckets (et non sur toute la map) : plusieurs threads peuvent écrire dans des buckets différents en même temps sans se bloquer mutuellement, et la structure interne reste toujours cohérente même sous forte concurrence.